

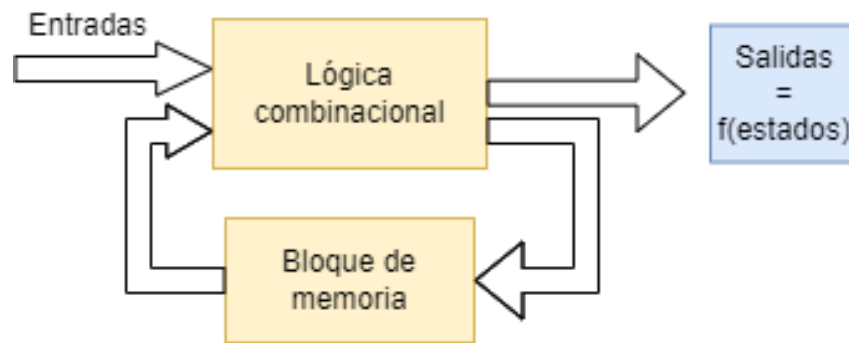
Introducción a VHDL

Matías Javier Oliva
Pablo García

Control: Máquinas de estado

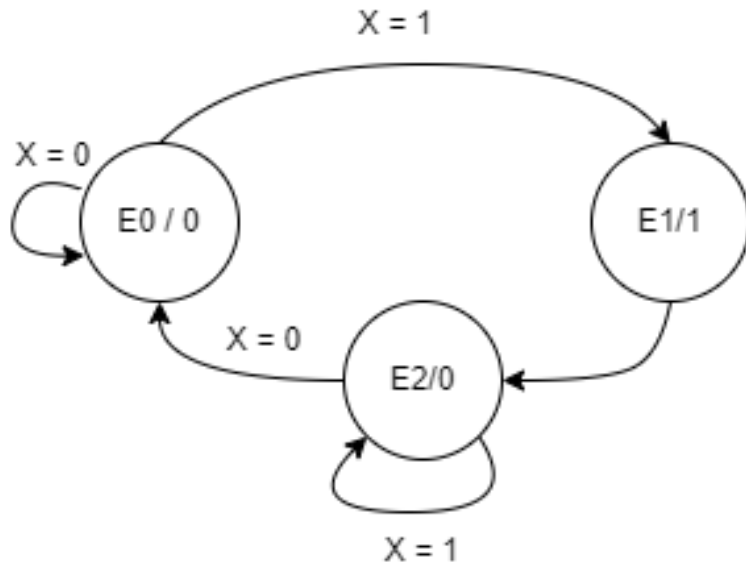
- Hasta ahora hemos visto como implementar elementos básicos de hardware en VHDL:
 - **Circuitos combinacionales simples:** Multiplexores, Barrel- shifters, Decodificadores.
 - **Circuitos secuenciales:** Distintos tipos de flip-flops, Contadores, Registros de desplazamiento.
 - **Circuitos aritméticos:** Sumadores, restadores y multiplicadores con distintas arquitecturas
- Pero todavía nos falta un elemento esencial para configurar un sistema funcional: El control de todos estos elementos aparentemente desconectados.
- Este control se puede modelar con la abstracción de las máquinas de estado.
- Esto es lo que en un procesador hace el CPU, que puede verse como una máquina de estado gigante.

Máquinas de estado: Modelo de Moore



- La idea de esta herramienta es que el sistema puede estar en una serie finito de estados, cada uno con ciertas características.
 - El sistema cambia de estado según su estado anterior y sus entradas.
 - Basado en estos estados controla una serie de salidas.
-
- Las salidas solo dependen del estado actual del sistema.
 - Mientras el sistema esté en un determinado estado la salida no puede cambiar.
 - La síntesis por este medio es en general más fácil de implementar que el método de Mealy pero usa más estados (más lógica).

Modelo de moore: ejemplo



- Para modelar sistemas con máquinas de estado son muy útiles los diagramas de estados.
- Se definen los estados que necesita el sistema.
- Como es modelo Mealy cada estado tiene directamente asociado una salida (por eso la notación Estado/Salida)
- En cada ciclo de reloj se evalúa la entrada y se cambia de estado siguiendo la línea correspondiente en el diagrama.
- Este ejemplo es un detector de flancos ascendentes de la entrada X.
 - E0: Entrada en 0 hace algún tiempo. Salida en 0.
 - E1: La entrada se pone en 1. Salida en 1 por un ciclo de reloj.
 - E2: Se espera a que la entrada vuelva a 0 antes de volver a E0. Salida en 0

Implementación en VHDL

- Para modelar máquinas de estado en VHDL es útil el constructo type.
- Esto genera un nuevo tipo de dato en VHDL que tiene solo una serie de valores posibles, a los que se le pueden poner nombres:

```
-- Tipo de estado nuevo con conjunto finito de valores
type state_type is (s0, s1, s2, s3);

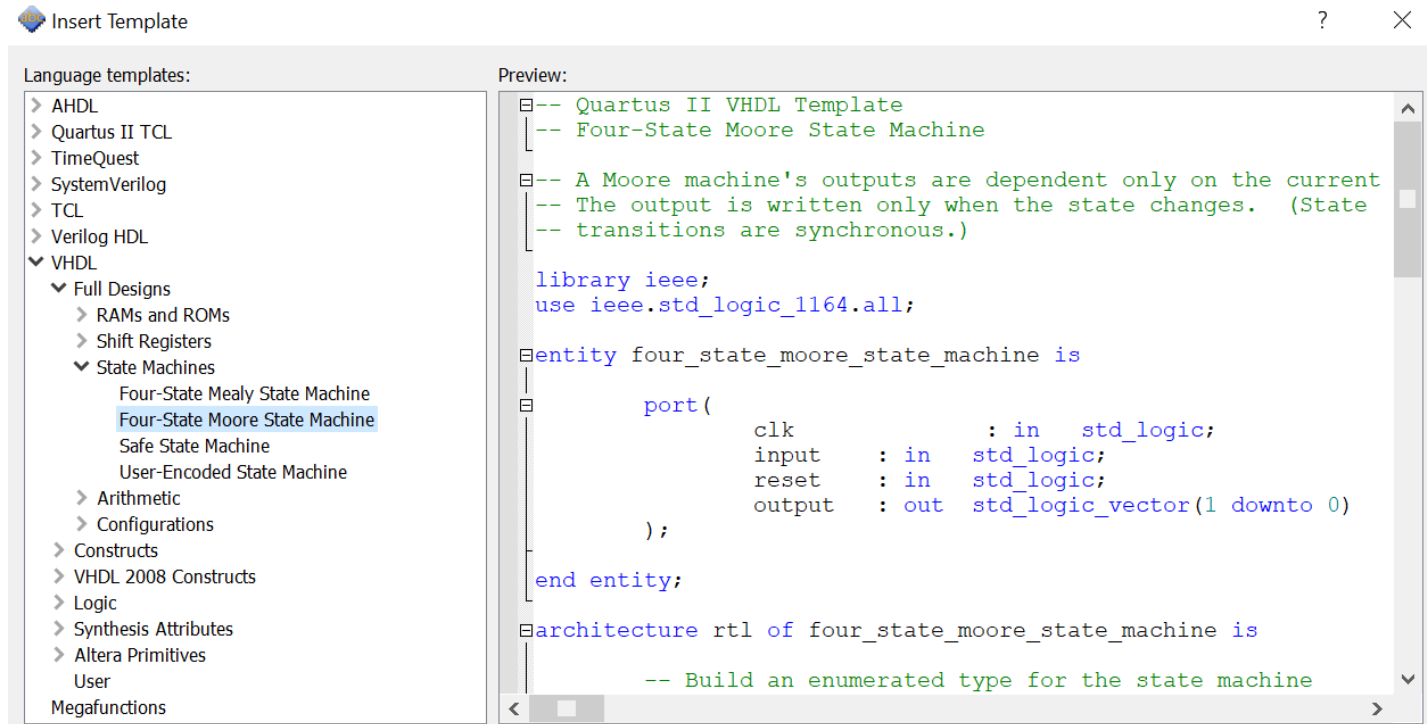
-- Registro de tipo state_type que guarda el estado actual
signal state : state_type;
```

- Luego el registro “state” de tipo state_type se puede asignar directamente usando el nombre de cada estado, abstrayéndonos del valor numérico que usa VHDL para implementar este estado internamente

```
if entrada = '1' then
    state <= s1;
else
    state <= s0;
end if;
```

Template de VHDL: Error común

- Para máquinas de estado se pueden usar los templates de VHDL:



- Pero con una salvedad: si se dejan los nombres input y output como entradas y salidas hay problemas en la simulación, ya que estas son palabras reservadas de Verilog

Implementación en VHDL

```
library ieee;
use ieee.std_logic_1164.all;

entity Clase6_a is

    port(
        clk      : in std_logic;
        entrada  : in std_logic;
        reset    : in std_logic;
        salidas   : out std_logic
    );

end entity;

architecture rtl of Clase6_a is

    -- Tipo de estado nuevo con conjunto finito de valores
    type state_type is (entradaEn0, Recibil, entradaEn1);

    -- Registro de tipo state_type que guarda el estado actual
    signal state : state_type;
```

Implementación en VHDL

```
-- Logica para avanzar al siguiente estado
process (clk, reset)
begin
    if reset = '1' then
        state <= entradaEn0;

    elsif (rising_edge(clk)) then
        case state is

            when entradaEn0=>
                if entrada = '1' then
                    state <= Recibil;
                else
                    state <= entradaEn0;
                end if;

            when Recibil=>
                state <= entradaEn1;

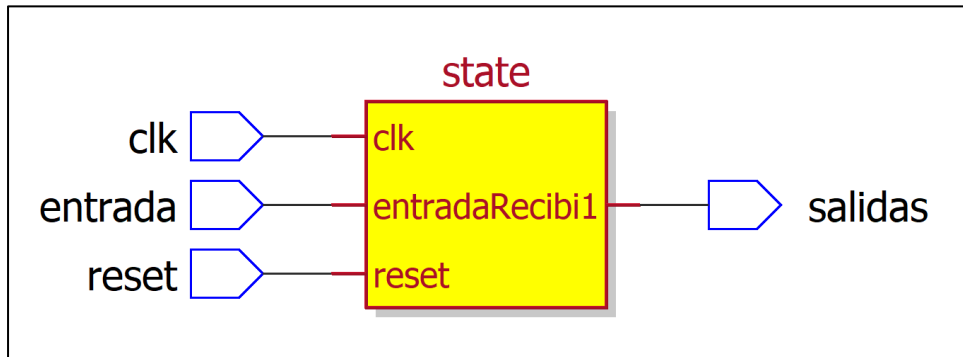
            when entradaEn1=>
                if entrada = '0' then
                    state <= entradaEn0;
                else
                    state <= entradaEn1;
                end if;

        end case;

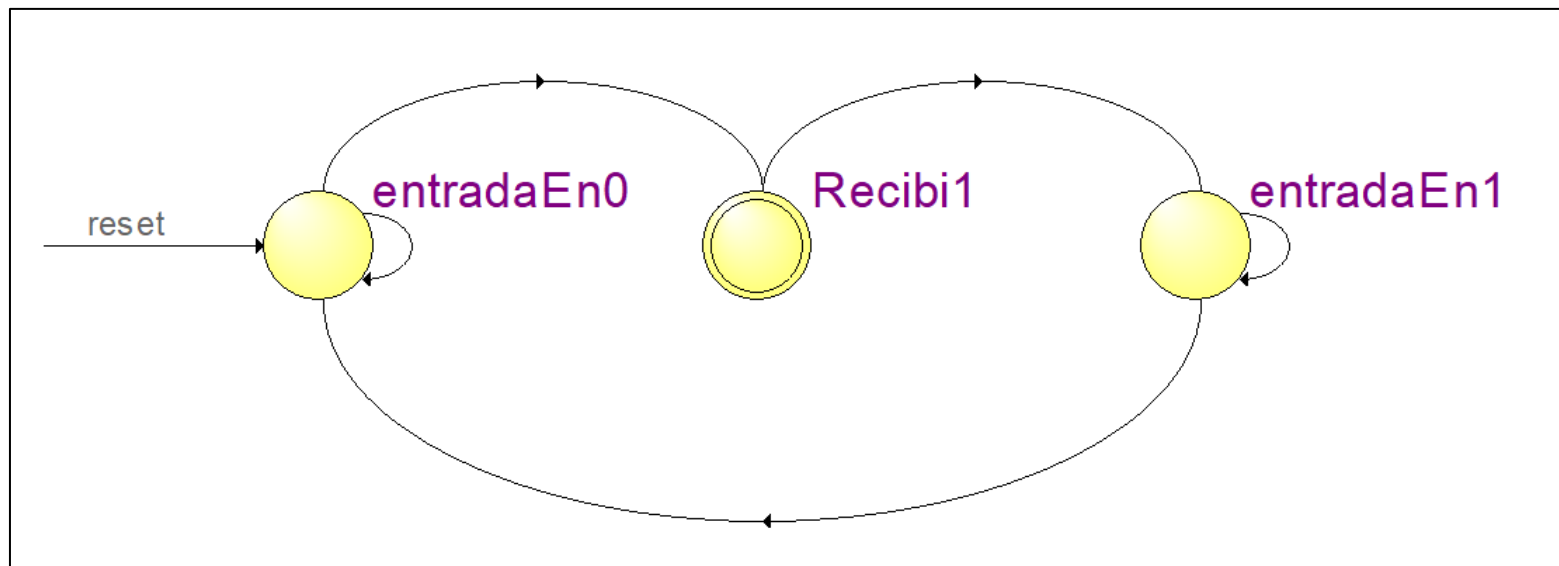
    end if;
end process;
```

```
-- La salida solo depende del estado actual
-- Es una lógica combinatoria entre estados
process (state)
begin
    case state is
        when entradaEn0 =>
            salidas <= '0';
        when recibil =>
            salidas <= '1';
        when entradaEn1 =>
            salidas <= '0';
    end case;
end process;
```

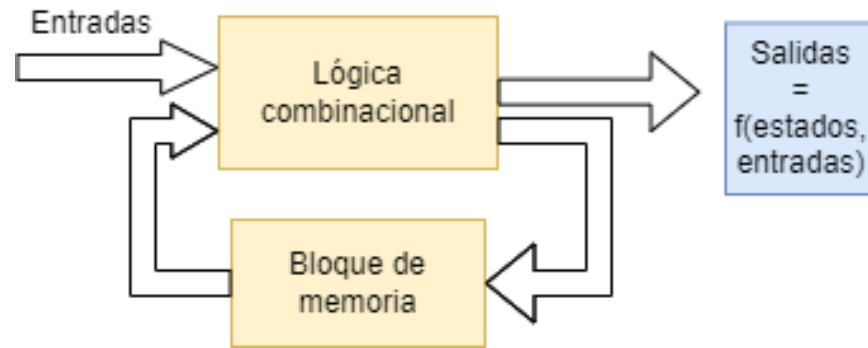

Diagrama RTL: ME de Moore



- El diagrama RTL ahora muestra una máquina de estados.
- Con el visor de máquinas de estado de Quartus se puede ver la implementación

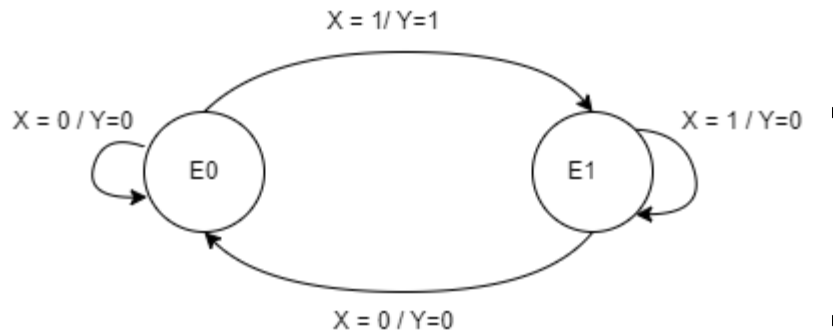


Máquinas de estado: Modelo de Mealy



- En el modelo de Mealy las salidas dependen de los estados actuales del sistema y además de las entradas.
- Las salidas pueden cambiar sin que cambie el estado (respondiendo a algún cambio en las entradas del sistema).
- En general la implementación de sistemas con el modelo de Mealy lleva a sistemas con menor cantidad de estados (menos lógica).

Máquinas de estado: Modelo de Mealy



- Ahora las salidas dependen de las entradas y de los estados.
- Por este motivo las salidas se ponen en las transiciones de estados.
- Estas se hacen con la misma lógica de las máquinas de Moore
- El mismo ejemplo anterior ahora puede sintetizarse con un estado menos; tenemos:
 - E0: La entrada está en 1.
 - E1: La entrada está en 0
- Aprovechamos el cambio en la entrada para cambiar la salida.
- Se disminuyen los estados pero como el cambio en la entrada puede ser asincrónico puede tener problemas de temporización.

Modelo de Mealy: Implementación

```
library ieee;
use ieee.std_logic_1164.all;

entity Clase6_b is

    port
    (
        clk      : in std_logic;
        entrada  : in std_logic;
        reset    : in std_logic;
        salida    : out std_logic
    );

end entity;

architecture rtl of Clase6_b is

    -- Build an enumerated type for the state machine
    type state_type is (s0, s1);

    -- Register to hold the current state
    signal state : state_type;
```

Modelo de Mealy: Implementación

```

process (clk, reset)
begin

    if reset = '1' then
        state <= s0;

    elsif (rising_edge(clk)) then

        -- El estado siguiente se determina
        -- en forma sincrónica según la entrada
        case state is
            when s0=>
                if entrada = '1' then
                    state <= s1;
                else
                    state <= s0;
                end if;
            when s1=>
                if entrada = '0' then
                    state <= s0;
                else
                    state <= s1;
                end if;
            end case;

        end if;
    end process;

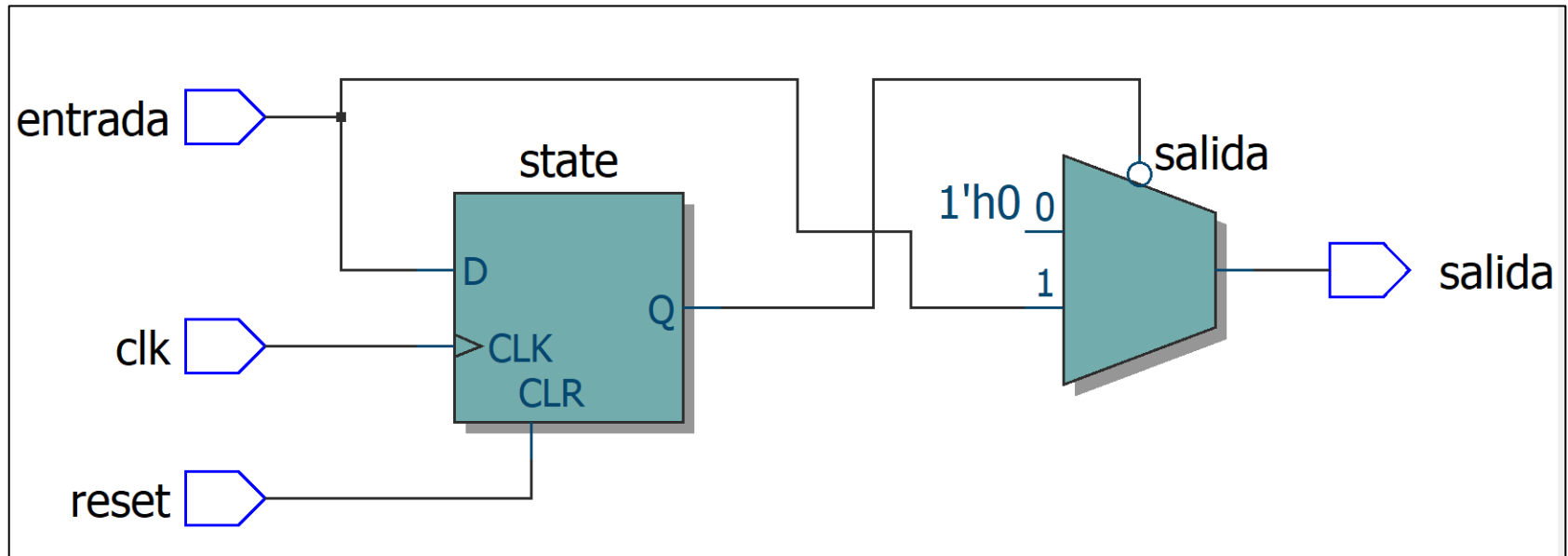
```

```

-- Determinamos la salida basada en el estado
-- actual y la entrada (sin esperar flanco de clk)
process (state, entrada)
begin
    case state is
        when s0=>
            if entrada = '1' then
                salida <= '1';
            else
                salida <= '0';
            end if;
        when s1=>
            if entrada = '1' then
                salida <= '0';
            else
                salida <= '0';
            end if;
        end case;
    end process;

```

Modelo de Mealy: RTL

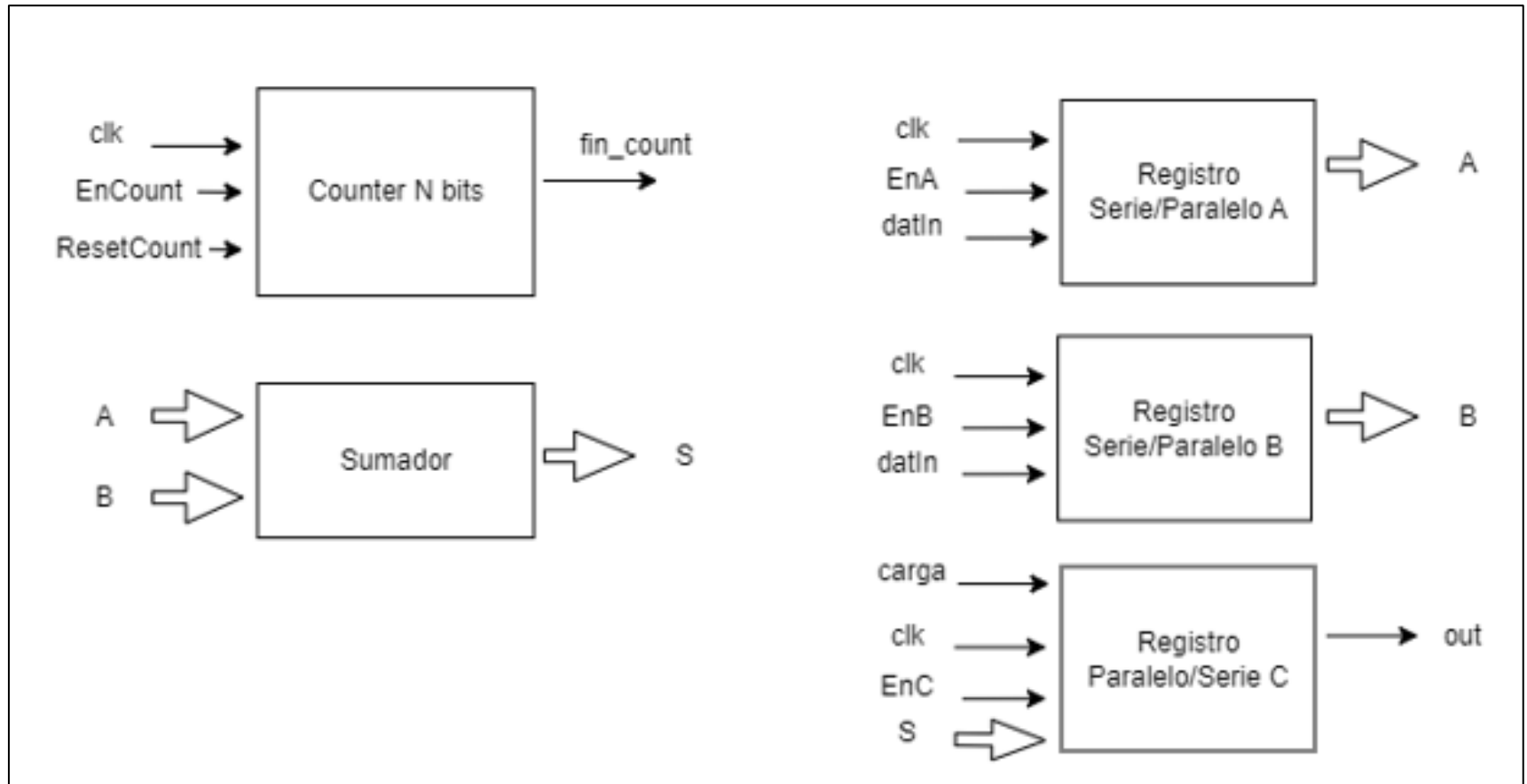


- En este caso Quartus pudo implementar el sistema con un único Flip-flop (alcanza para gestionar dos estados).
- Por este motivo no muestra una máquina de estados en el visor de máquinas de estado.

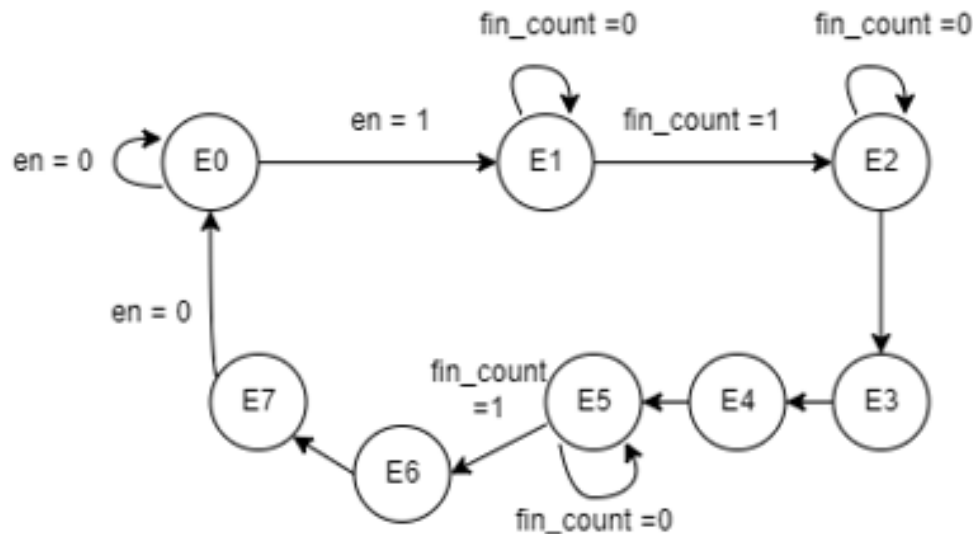
Control de sistemas con ME

- Si bien estos ejemplos son sencillos y describen funcionalidades que podrían inferirse sin el uso de las ME estas resultan un elemento fundamental para diseñar el control de sistemas.
- Pensemos por ejemplo en un sistema que recibe en forma serie dos operandos (uno seguido de otro), luego los suma y finalmente los transmite en forma serie.
- Para un sistema como este necesitamos dos registros de desplazamiento Serie/Paralelo, un sumador, un registro de desplazamiento Paralelo/Serie y posiblemente un contador.
- Gestionar todos estos elementos por separado resulta dificultoso sin la herramienta de las ME. En cambio usandolas se puede separar las estructuras del control, simplificando la operación.

Control de sistemas con ME



Control de sistemas con ME



E0: Esperara a que se habilite la operación. Salidas en 0 (EnA, EnCount, ResetCount, EnB, EnC)

E1: Se carga el registro A. N ciclos de reloj. EnA = 1. EnCount = 1. ResetCount = 0.

E2: Se carga el registro B. N ciclos de reloj. EnB = 1. EnCount = 1. ResetCount = 0.

E3: Se hace la suma. EnSum = 1 EnCount = 0. ResetCount = 1.

E4: Habilita carga del registro C.

E5: Se transmite la salida. N ciclos de reloj. EnC = 1. EnCount = 1. ResetCount = 0.

E6 y E7: Estados extras para sincronizacion

Implementación por Moore

Implementación

```

counter: contador_asinc generic map (log2N) port map ( clk, resetCount ,enCount,fin_count, open );
registroA: RegistroSerieParalelo generic map (N) port map (clk,reset,'0',enA,dat_in,regA);
registroB: RegistroSerieParalelo generic map (N) port map (clk,reset,'0',enB,dat_in,regB);
sumador: RippleCarry generic map (N) port map (regA,regB,regS);
registroC: RegistroDesplazamiento generic map (N) port map (clk,reset,'0',enC,cargaRegC,regS,dat_out,dat_out_valid);
ME : MaquinaEstados port map (clk,reset,enable,fin_count,enA,enB,enC,enCount,ResetCount,cargaRegC);

```

```

library ieee;
use ieee.std_logic_1164.all;

entity MaquinaEstados is
port
(
    clk : in std_logic;
    reset : in std_logic;
    enable : in std_logic;
    fin_count : in std_logic;

    enA : out std_logic;
    enB : out std_logic;
    enC : out std_logic;
    enCount : out std_logic;
    ResetCount : out std_logic;
    cargaC : out std_logic
);
end entity;

```

```

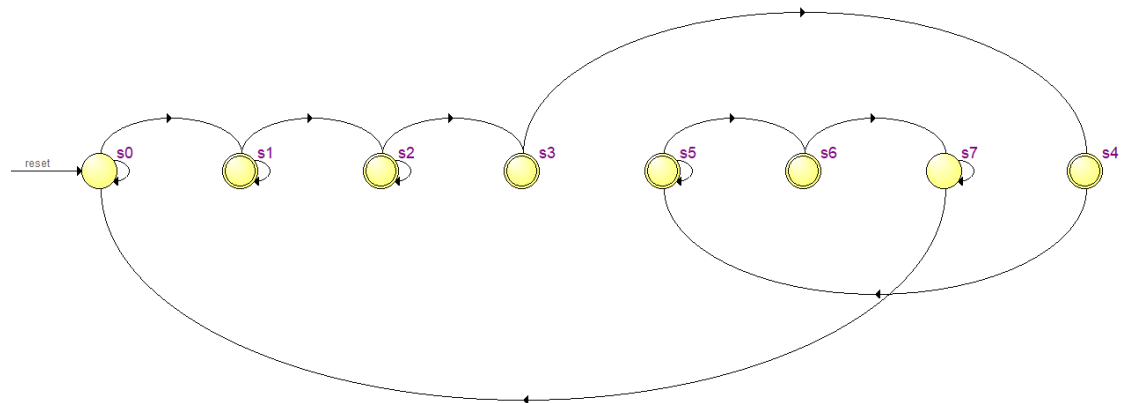
architecture rtl of MaquinaEstados is

    -- Build an enumerated type for the state machine
    type state_type is (s0, s1, s2, s3, s4, s5, s6,s7);

    -- Register to hold the current state
    signal state : state_type;

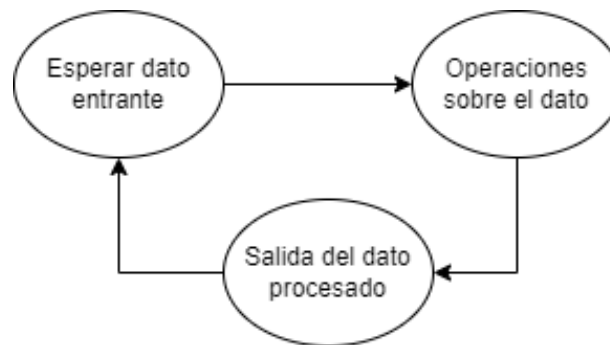
    signal salidas : std_logic_vector (5 downto 0);
    -- salidas = enA & enB & enC & enCount & ResetCount & cargaC

```



Implementación de pipelines

- Cuando se trabajan con datos entrantes una opción para gestionarlos es utilizando una maquina de estados del tipo:



- Pero esto tiene algunas limitaciones:
 - Se tardan (al menos) 3 ciclos de reloj en gestionar un dato.
 - Si los datos llegan de a un dato por ciclo de reloj es impracticable
- En cambio se podría gestionar el sistema implementando un pipeline, donde se hacen las operaciones sobre el dato mientras se espera la siguiente muestra de señal.

Implementación de pipelines

- La naturaleza paralela del diseño de hardware hace que esto sea realmente muy fácil de implementar:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Clase6_d is
  generic( N : integer := 32 );
  port( clk: in std_logic;
        reset : in std_logic;
        dat_in: in std_logic_vector (N-1 downto 0);

        dat_aux0 : out std_logic_vector (N-1 downto 0);
        dat_aux1 : out std_logic_vector (N-1 downto 0);
        dat_aux2 : out std_logic_vector (N-1 downto 0);

        dat_out: out std_logic_vector (N-1 downto 0) );
end entity;
```

Implementación de pipelines

```

architecture arch of Clase6_d is
  signal dat_0, dat_1, dat_2 : std_logic_vector(N-1 downto 0);
begin
  process (clk) is begin
    if (reset = '1') then
      dat_0 <= (others => '0');
      dat_1 <= (others => '0');
      dat_2 <= (others => '0');

    elsif (rising_edge(clk)) then
      dat_0 <= dat_in;

      dat_1 <= std_logic_vector ( unsigned(dat_0) + 15 ) ;

      dat_2 <= std_logic_vector ( unsigned(dat_1) + 42 ) ;

    end if;
  end process;

  dat_out <= dat_2;

  dat_aux0 <= dat_0;
  dat_aux1 <= dat_1;
  dat_aux2 <= dat_2;

end arch;

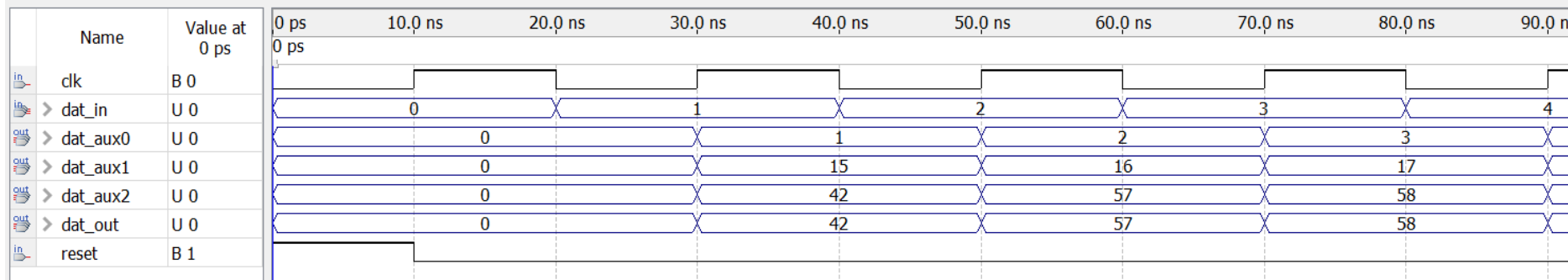
```

En este ejemplo se hacen algunas operaciones genéricas sobre los datos que van entrando.

3 flip-flops (que se instancian implícitamente) implementan el pipeline.

Se agregaron señales auxiliares para ver las salidas en una simulación rápida

Implementación de pipelines



- El pipeline introduce una latencia de 3 ciclos de reloj frente a los datos entrantes.
- Con estas estrategias se pueden hacer operaciones en forma más rápida a costa de una latencia, y se pueden gestionar datos que entran de a uno por ciclo de reloj.
- Esto es interesante al comparar con soluciones implementadas en software con microprocesadores por ejemplo.